Decode-Before-Guard Compression for UAV Energy Telemetry

Phan Tan Nhat Truong[1⋆] and Bui Thi Thanh Ngan[2,3*]

[1] FPT University, Ho Chi Minh City, Vietnam
[2] University of Social Sciences and Humanities, Ho Chi Minh City, Vietnam
[3] Viet Nam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam
truongptnse200033@fpt.edu.vn, buithithanhngan0311@gmail.com

Abstract. UAV energy telemetry must be transmitted efficiently while preserving both short-term power variations and cumulative energy consumption. The proposed streaming compression method decodes each candidate packet before evaluating power error, prefix-energy error, nonnegativity and segment duration. Absolute-energy checkpoints and explicit wire-format fields ensure that the guarantees apply to the reconstructed signal received by the decoder. Experiments on public UAV flight telemetry show how quantization precision affects compression size and the proportion of records satisfying the error constraints. Offline alternatives are also compared while their different operational assumptions are kept explicit. The results provide a reproducible framework for compression with decoded-signal guarantees; sensor accuracy, erased-data recovery and radio-energy improvements remain outside the scope of validation.

Keywords: UAV telemetry, Streaming compression, Quantization, Energy accounting

1 Introduction

An unmanned aerial vehicle (UAV), commonly called a drone, can record its power use during flight. Power is the rate of energy use at one moment. Accumulated energy is the amount used since the record began. A smaller record should preserve short power changes and energy used over time.

Our method saves selected points and draws a curve between them after decoding, meaning reading the stored data back. Rounding can make this curve fall below zero between positive saved values. That fails our requirements because the reference power is nonnegative.

The main idea is to pack the values, read them back and check the recovered curve. This explains "decode before guard" in the title. A guard is a check, and the contract is the set of required checks. An explicit failure is reported when no permitted continuation passes.

Figure 1 separates the receiver, which has only transmitted data, from an audit that retains the original record. Experiments compare passing records, failure causes and bytes needed.

⋆ These authors contributed equally. Both are corresponding authors.

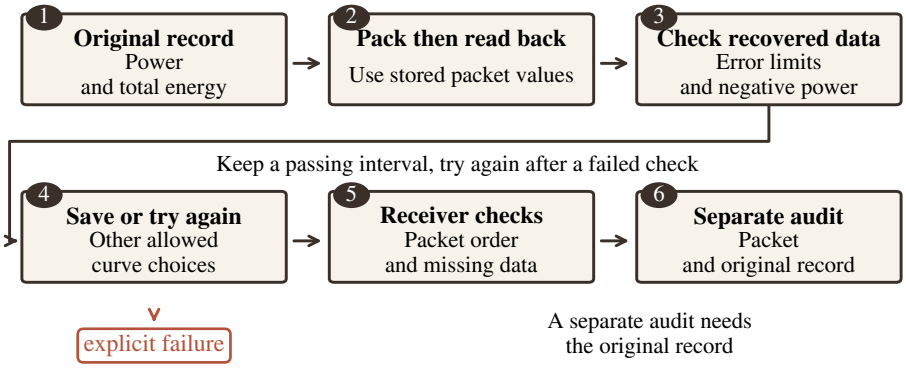


Fig. 1. The sender reads back the packed values before checking them, while a separate audit keeps the original record

2 Related Work

Existing compression methods keep different kinds of information. Lossless methods preserve every recorded value exactly. Examples include Gorilla [1], Sprintz [2] and Chimp [3]. Approximate methods allow differences within a chosen limit. SZ [4], ZFP [5] and SZ3 [24] provide such controls in their error-bounded settings.

Another approach replaces many measurements with line segments. Some methods process the full record, such as the simplification of Douglas and Peucker [6]. Others work as samples arrive, including connected online approximation [7] and Swing and Slide [18]. Sim-Piece groups similar segments [25]. Earlier work also optimizes segment choices [9] and constrains curve shape [8]. Our contribution combines checks on the actual decoded curve rather than introducing these individual ideas.

Rounding changes numerical calculations, as explained by Goldberg [10]. IEEE 754 defines floating-point representation and operations [11]. Those numerical rules do not establish that a physical sensor is accurate. Table 1 compares the requirements addressed here.

3 What a Compressed Record Must Preserve

The reference joins consecutive measurements with straight lines. It is called piecewise-linear, or PL. Area under it gives accumulated energy in watt-hours (Wh). This running value, also called “prefix energy”, is checked throughout the record.

Table 1. Different methods preserve different information

Approach	Recovered values	Energy and negative power	As data arrive
Lossless	Every value kept exactly	Preserved with the same reference[a]	Depends on the method
Error-bounded lossy	Sample errors stay within chosen limits	Separate checks needed	Depends on the method
Line-segment methods	Segment errors follow the chosen rule	Separate checks needed	Processes new samples
This study	Checked throughout each interval	Energy error and negative power checked	12 stored samples and a 5.5 s bound[b]

[a]The same nonnegative values and time points joined by straight lines. [b]Strict uses a 0.5 s sample step and a 5 s maximum interval. It can report an explicit failure. Other methods' failure behavior depends on their implementation.

A recovered interval must keep power error within 25 watts (W) and energy error within 1 mWh, remain nonnegative and last at most 5 s. One mWh is one-thousandth of a watt-hour. Error means difference from the reference. These targets are not validated flight-safety limits.

We seek fewer bytes under these conditions. Earlier choices can force failure as new samples arrive. An offline method, which sees the whole record first, may still succeed. Appendix A gives the equations and proofs.

4 How the Method Works

4.1 Save Points, Then Check the Curve Between Them

Each saved point, or checkpoint, contains time, power and energy accumulated so far. The decoder connects two checkpoints with a curve matching their powers and energy change. Matching that energy change alone does not control every point between them.

The sender checks the largest error throughout the decoded interval and whether power falls below zero. This includes rounding changes. An ending marker identifies the stream's final point. Packet details and calculations are in Appendix A.

4.2 Leave Room for Rounding Without Rejecting Valid Zero Power

Leaving part of the error allowance unused can cover rounding changes. Theorem 1 gives sufficient amounts. Table 2 shows that larger rounding steps need more room.

Table 2. Larger rounding steps need more room within the error limit

Rounding	Room for rounding power (W)	Room for rounding energy (mWh)	Energy budget used (%)
Fine	0.005	0.0018	0.18
Medium	0.05	0.0179	1.79
Coarse	0.5	0.1786	17.86

These are upper bounds on the extra error that rounding can add. They are not measured energy losses. The limits are 25 W and 1 mWh. Values are rounded.

MarginErr reserves room for power and energy error, then checks negative power directly after decoding. FullMargin also requires room above zero and can reject a valid curve that touches zero. Comparing them reveals the cost of that stronger rule.

4.3 When Two Positive Endpoints Can Hide a Negative Valley

Too little stored energy for the endpoint powers forces the curve downward and may make it cross zero. Endpoint checks would miss this valley. Lemma 1 gives the exact condition, illustrated in Figure 3.

4.4 What Happens When a Check Fails

Strict keeps extending an interval while the decoded curve passes. After a failed extension, it saves the last passing point and restarts there. If even the adjacent interval fails, it reports failure, called refusal in the results.

Search tries up to nine nearby rounded pairs. Escape can try a flagged straight line under the same checks. TwoPiece joins two lines through a middle point to preserve area and nonnegative power. Combining TwoPiece with Escape gives the hybrid.

Ties prefer fewer bytes, then smaller energy error, power error and changes to rounded counts. Energy always uses the same origin, so restarting an interval does not reset its error allowance.

Strict holds at most 12 samples with a 5.5 s release bound. Longer lookahead costs memory and delay. Horizon2 needs 13 samples and 6 s, while Horizon4 needs 15 samples and 7 s. An offline comparator finds the cheapest sequence of allowed rounded checkpoints.

5 What the Receiver Can Verify

The receiver lacks discarded measurements. It combines packet numbers and elapsed time to detect missing saved points, because numbering eventually starts again. An interval is certified only when its received endpoints were uniquely identifiable as neighbors before transmission. Other gaps remain uncertain.

Table 3 shows why packet consistency cannot establish agreement with discarded measurements.

Algorithm 1 Read back a proposed interval before deciding to keep it

Input: time-stamped power measurements, error limits, rounding settings and the permitted recovery options.

1 Start the record and its first saved point. Set accumulated reference energy to zero.
2 Read the next measurement and update the energy used so far.
3 Propose a new saved point, round its values, pack them and read them back. Search may try up to nine nearby pairs.
4 Check power error, accumulated-energy error, negative power and interval length. Keep extending while a permitted proposal passes.
5 After a failed extension, save the last passing point and retry from there. If the adjacent continuation also fails, try an allowed straight-line escape. Report failure if no permitted continuation passes.
6 At the end, save any pending accepted point and add the ending marker.

Output: a completed encoded record or an explicit failure, including the bytes already emitted.

Table 3. Received data can be checked for consistency, while agreement with the original requires more information

What can be checked	What remains unknown
Packet structure and time order	Whether the original sensors were accurate
Whether two saved points were neighbors	A unique history when several histories fit
Negative power and excessive duration	Error against measurements no longer available
The ending marker, if received	Whether the ending is complete if the marker is lost
Checksums for accidental corruption	Authenticity and the contents of missing packets
An occasional check inside an interval	The largest error anywhere in that interval

A middle-of-interval spot check observed a median of about 46% of the true maximum error. Appendix B instead uses the full reference to check the whole curve.

6 How the Comparison Was Set Up

IDF [12] and its archive [13], CMU [14] and KiltHub [15], and AMOVFLY [16] provide 706 records. Solar [17] adds twelve descriptive windows from one campaign. Preparation removes duplicate times, splits long gaps and places eligible prefixes on a half-second grid for up to two minutes.

Dense stores the full prepared record. Using 20% of Dense means one fifth of its bytes. Failures cost their partial output plus a Dense restart. ZFP, zstd [19] and SZ3 face the same decoded checks.

The native comparison uses each method's own format. A second comparison puts recovered checkpoints in one shared format. Native32 uses full-precision checkpoints, Iso searches limited error settings, FloatDeltaZstd compresses individually stored samples and Sim-Piece searches small or expanded sets of settings.

Comparisons use the same records. Records collected together stay grouped when estimating uncertainty because they may share conditions. Appendix C gives details. The illustrative one-percentage-point threshold was chosen during revision and still needs validation by operational users.

7 What the Results Mean

7.1 Different Checks Prevent Different Failures

PreRound checks the proposed curve before packet rounding. Most of its observed violations concern accumulated energy. All original Strict first refusals concern the reconstructed power falling below zero. Table 4 counts these different failure groups. The evidence supports keeping both energy and nonnegative-power checks.

Table 4. The two controls fail for different reasons

Failed check	PreRound 39 records	Strict 21 first refusals
Power falls below zero	2	21
Power error is too large	1	0
Accumulated energy error is too large	36	0
Segment is too long	0	0

PreRound checks before rounding, while Strict checks decoded values. Strict counts original first refusals across precision profiles.

Figure 2 follows an AMOVFLY record selected by reference variation, not favorable error. Its lower panel divides errors by their limits. Absolute values above one would exceed a limit.

7.2 A Passing Record and a Smaller File Are Separate Outcomes

In Table 5, first read passing records, then bytes. MarginErr matches Strict at 702 of 706 without violations, providing a fairer comparator than FullMargin. Escape passes all 706. Search passes 700 prefixes but only 205 of 270 full AMOVFLY records, so short-record success does not guarantee full-record success.

ZFP remains smaller natively, using 14.4% of Dense against Escape's 17.5%. Converting samples into checkpoints changes what is stored and cannot reverse that result. Those entries read "Different format". On a nominal 32-byte-per-second payload, 17.5% is about 5.6 bytes per second before framing and protection. This measures bytes, not radio energy.

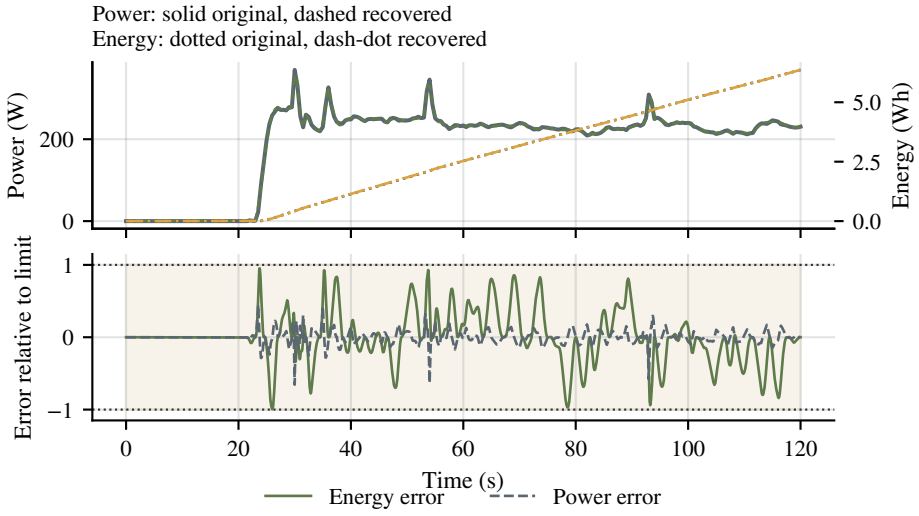


Fig. 2. One record shows recovered power and energy, with errors relative to their limits

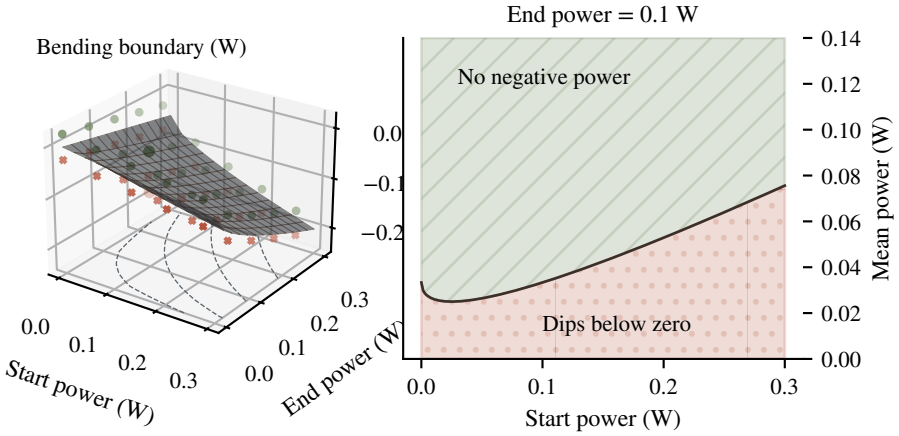


Fig. 3. The mathematical boundary shows when power between saved points falls below zero

Table 5. Read the number of passing records before comparing file sizes

Method	Records passing out of 706	Transmitted bytes (%)	Shared-format bytes (%)
Escape	706/706	17.50	31.50
MarginErr	702/706	17.96	31.91
FullMargin	360/706	55.65	64.89
PreRound	667/706	22.66	36.65
TwoPiece	706/706	17.21	31.78
Native32	706/706	37.10	31.50
ZFPIso	706/706	14.40	Different format
zstd	706/706	59.30	Different format
FloatDeltaZstd	706/706	25.29	Different format
SimPiece5Grid	702/706	21.51	Different format
SimPiece33Grid	706/706	18.74	Different format

Dense is the full-record size reference. Costs include failures and restart. Shared-format bytes use common checkpoints. *Different format* identifies dense expansion that changes the comparison. Actual file sizes remain available. Values are rounded.

7.3 Replacing Negative Power by Zero Repairs Only Part of the Problem

Clipping replaces negative power with zero. This reduces error at each point against a nonnegative reference but adds area. Original worst energy error plus that area bounds the corrected energy error.

Both tested failure groups remain below 0.015 mWh, against a 1 mWh allowance. The changed endpoint energy still prevents a conclusion about full-record recovery or the necessity of TwoPiece or Escape.

7.4 Small Savings Depend on the Storage Format

TwoPiece passes 718 prefixes but fails on one full episode. The hybrid completes every tested episode, as Escape already does. Its benefit over Escape concerns bytes and curve behavior. Figure 4 shows additional options handling the harder intervals.

Table 6 reports changes from Escape. Negative means fewer bytes, positive more bytes, and brackets give uncertainty. An interval spanning zero allows either direction. Percentage points subtract two percentages rather than giving a relative reduction.

All mean effects are below one percentage point. Native savings coexist with higher shared-format costs on IDF and AMOVFLY, while CMU allows either direction. Small p values do not establish a large practical benefit.

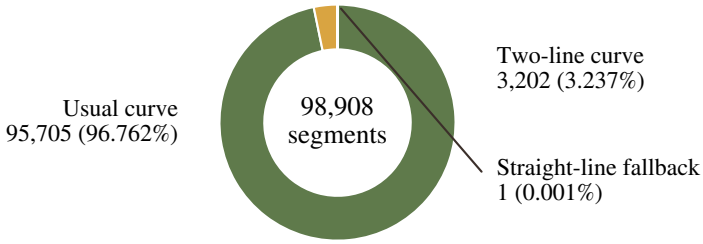


Fig. 4. Most intervals use the usual curve, while two-line curves and one straight-line fallback handle harder cases

Table 6. The size advantage depends on how the output is stored

Scope	Source	Transmitted bytes difference [95% interval]	Shared-format bytes difference [95% interval]
120 s	IDF	-0.06 [-0.08, -0.04]	0.44 [0.33, 0.56]
120 s	CMU	-0.62 [-0.76, -0.49]	0.00 [-0.32, 0.31]
120 s	AMOVFLY	-0.13 [-0.16, -0.10]	0.44 [0.35, 0.54]
Full episode	IDF	-0.09 [-0.11, -0.06]	0.75 [0.65, 0.86]
Full episode	CMU	-0.57 [-0.68, -0.45]	0.04 [-0.27, 0.34]
Full episode	AMOVFLY	-0.25 [-0.43, -0.13]	0.65 [0.50, 0.77]

Changes from Escape are percentage points. Negative indicates savings. Brackets give 95% group-based intervals. Prefixes use TwoPiece, full episodes the hybrid. Adjusted p is 0.000120, or 1.000000 for CMU in the shared format. Values are rounded.

On the declared IDF prefix choices, gaps to the offline comparator are 4.7% for Strict, 1.9% for Horizon2 and 1.5% for Horizon4. Extra lookahead reduces this gap at the stated memory and delay cost.

7.5 Missing the Final Marker Leaves the Ending Uncertain

In the tested independent-loss setting, terminal-erasure detection is perfect when the ending marker arrives. Including possible marker loss reduces the average to about 89%. A missing marker leaves the receiver unable to establish the ending. Appendix D compares recovery options and their byte costs. Solar remains descriptive, with full-hybrid windows capped at 30 minutes.

8 What These Results Do Not Establish

Preparation differences can exceed the error limits. Passing means agreement with the prepared reference, without validating sensors. Controls and the interpretation threshold were added during revision.

The exact audit covers ideal curves, not device rounding or number-field overflow. Five linear escapes have nonzero join-energy differences below tolerance. After warm-up, Strict takes about 30 microseconds per sample on the tested computer. This is not an onboard timing guarantee. Future-flight reliability and radio-energy savings remain unvalidated.

Source data use the cited archives. Earlier code and results are at `https://github.com/truong16102006-lgtm/uav-decode-before-guard-audit-v8-20261030`, which does not yet include this revision. Generative AI assisted manuscript preparation and technical checks. The authors take responsibility for the final content.

9 Conclusion

Reading back packed values exposes distinct energy and negative-power failures. Fair rounding reserves match Strict's completion, while clipping repairs local errors but changes endpoint energy. ZFP's smaller native files and the hybrid's small effects limit compression claims. The checks concern a prepared reference, with physical and operational validation still needed.

A Mathematical Details

These equations and proofs use the prepared reference and declared rounding rules.

Stored values and the reconstructed curve The 36-byte header stores origin, sample step and rounding increments. Shortest unsigned LEB128 [27] stores integer time, power and energy counts. Normal packets add three bookkeeping bytes. Escapes store full-precision values, and a footer marks the ending. Invalid fields, clocks and overflow cause failure. Medium rounding steps are 0.1 W and 0.001 mWh. Fine steps are ten times smaller and coarse steps ten times larger.

For decoded endpoints (t_a, p_a, E_a) and (t_b, p_b, E_b), write $D = t_b - t_a > 0$ and $u = (t - t_a)/D$. The conservative quadratic and its integral are

$$c = \frac{3600(E_b - E_a)}{D} - \frac{p_a + p_b}{2}, \tag{1}$$

$$Q(u) = p_a + (p_b - p_a)u + 6cu(1 - u), \tag{2}$$

$$\widehat{E}(u) = E_a + \frac{D}{3600}\left[p_a u + \frac{p_b - p_a + 6c}{2}u^2 - 2cu^3\right]. \tag{3}$$

Endpoints and stationary points locate the largest errors. The bounds are

$$e_P = \|Q - P\|_\infty \le 25 \text{ W}, \quad e_E = \|\widehat{E} - E\|_\infty \le .001 \text{ Wh}, \quad q_{\min} \ge 0, \quad D \le 5 \text{ s}. \tag{4}$$

The infinity norm gives the largest absolute error. Two retained sets of 1,200 intervals cross-check the calculation.

Allowance reserved for rounding

Theorem 1 (Finite-duration rounding margins). *For fixed endpoint times with $h \le D \le L$, perturbations bounded by q_P W and q_E Wh obey*

$$\|\delta\widehat{E}\|_\infty \le q_E + q_P D/14400 = m_E(D) \le m_E(L), \tag{5}$$

$$\|\delta Q\|_\infty \le \max\{q_P, q_P/2 + 10800 q_E/D\} = m_P(D) \le m_P(h). \tag{6}$$

Thus pre-rounding errors below $B_P - m_P(D)$ and $B_E - m_E(D)$ are sufficient for decoded error bounds. Decoded nonnegativity must either be checked directly or guaranteed by the stronger pre-rounding condition $q_{\min} \ge m_P(D)$.

Proof. The endpoint-energy basis functions are nonnegative and sum to one, while the absolute endpoint-power coefficients in $\widehat{E}$ sum to at most $D/14400$. Differentiation gives power coefficients $1 - 4u + 3u^2$, $-2u + 3u^2$ and opposite energy coefficients $21600u(1-u)/D$. Maximizing their absolute envelope gives the stated power bound, and triangle inequalities give the error margins. Their monotonic dependence on D yields the finite constants.

When the reconstructed power stays nonnegative

Lemma 1 (Shape feasibility of a decoded quadratic). *For nonnegative endpoint powers p_a, p_b and positive duration D, the reconstructed power stays nonnegative throughout the segment if and only if*

$$c \ge -\frac{(\sqrt{p_a} + \sqrt{p_b})^2}{6}. \tag{7}$$

Proof. Write $Q = p_a(1-u)^2 + p_b u^2 + (p_a + p_b + 6c)u(1-u)$. The arithmetic–geometric mean inequality gives the necessary and sufficient condition $p_a + p_b + 6c \ge -2\sqrt{p_a p_b}$, including zero-endpoint limits. Rearranging gives the stated bound.

A constant-cost vertex check agrees on 437,934 candidates, including 49 negative cases. Candidate agreement does not establish full-stream completion.

B Independent Check with Exact Arithmetic

A separate program keeps the reference and encloses the recovered curve within a known range. Wide, undecisive intervals are divided further using exact fractions and the Bernstein basis [26]. Coefficients between one and two enclose the curve in that range.

PASS proves compliance. FAIL gives a violating point. UNKNOWN means subdivision was insufficient and is never a pass. The following proof justifies decisive answers.

Lemma 2 (Sound integer-subdivision certificate). *A Bernstein polynomial stays between its smallest and largest coefficients. Positive integer scaling and dyadic subdivision preserve this enclosure. PASS requires all leaves within bounds. FAIL requires an exact endpoint violation. UNKNOWN leaves a case unresolved at depth 12.*

Proof. Bernstein weights form a convex combination. Subdivision averages the coefficients, and positive integer scaling preserves their signs. The enclosure therefore remains valid after subdivision. An exact violating endpoint proves failure. An unresolved enclosure leaves both acceptance and failure unproven.

All audited real-data intervals pass with declared and zero added tolerances, including 98,908 full-hybrid intervals. Of 4,000 deliberately difficult constructed cases, 2,550 remain UNKNOWN, including a narrow negative valley. Decisive answers are correct, without a fixed-width hardware guarantee.

C Sensitivity and Statistical Details

Cluster bootstrap [20] intervals resample 38 IDF recording groups, 18 CMU dates and 21 AMOVFLY dates. The twelve effects use 10,000 bootstrap replicates, 100,000 sign flips under group sign symmetry and a separate Holm adjustment [21] for multiple comparisons. The one-percentage-point interpretation threshold was not preregistered.

Table 7 asks what changes when more energy error is allowed. Within a row, the file becomes smaller while completion is unchanged. Changing the time between samples also changes the reference curve, so comparisons across sample steps have different fidelity targets.

Table 7. Allowing more energy error makes files smaller but leaves some failures

Time between samples (s)	Longest interval (s)	Completed	Tight error 0.5 mWh	Default error 1 mWh	Looser error 2 mWh
0.5	2.5	702/706	22.05	20.59	19.30
0.5	5	702/706	19.90	17.93	16.07
1	2.5	702/706	36.30	35.30	35.27
1	5	703/706	29.41	26.74	24.97

The last three columns give bytes as a percentage of Dense. Each row has the same completion across allowances. Changing the time between samples changes the comparison reference.

Table 8 summarizes 36 earlier comparisons, separate from the 42-comparison extension and twelve new effects.

Table 8. A detectable difference can still be a small saving

Source	Comparisons	Statistical test results below 0.05
IDF	9	8
CMU	9	9
AMOVFLY	9	9
Pooled non-Solar	9	8

P values are adjusted for multiple comparisons. Small values support a difference under the test assumptions, without measuring its size or direction.

Native32's shared-format intervals include zero, with adjusted p values above 0.6. This does not establish native equivalence. Coarser precision reduces bytes on the shared 685-record subset. Neither energy-aware nor ordinary-Swing tests establish equivalence at the one-percentage-point margin. Absence of a detected difference is not equivalence. Walsh-median bootstrap intervals can be asymmetric, and simulated p values have finite resolution.

D Packet Loss and Recovery

Thirty repeats compare independently missing packets with Gilbert–Elliott runs of consecutive losses [22], averaging three packets per bad run [23]. The header is assumed delivered. Framing, ending markers and recovery information all count toward bytes.

Repetition sends endpoint information again. Parity recovery, called XOR in the implementation, adds information that can be used after reception. Packing sixteen saved points into one packet can add up to 75 s delay under the declared cap. Table 9 reports usable time together with the size cost.

Table 9. Recovery information keeps more of the record usable at an added size cost

Saved points per packet	Protection	Random packet loss usable time (%)	Consecutive losses usable time (%)	Bytes versus base stream
1	None	81.1 ± 0.3	86.6 ± 0.4	1.28
1	Repetition	97.9 ± 0.1	91.1 ± 0.4	2.17
1	Parity recovery	93.9 ± 0.3	88.9 ± 0.6	1.76
16	None	89.6 ± 0.9	89.6 ± 0.9	1.02
16	Repetition	90.2 ± 0.7	89.6 ± 1.0	1.07
16	Parity recovery	96.9 ± 0.5	91.7 ± 0.9	1.40

Usable intervals require boundary points that were neighbors before transmission. At 10% packet loss, entries show mean and standard deviation over 30 repeats. This spread describes repeat variability. Byte factors multiply base-stream bytes.

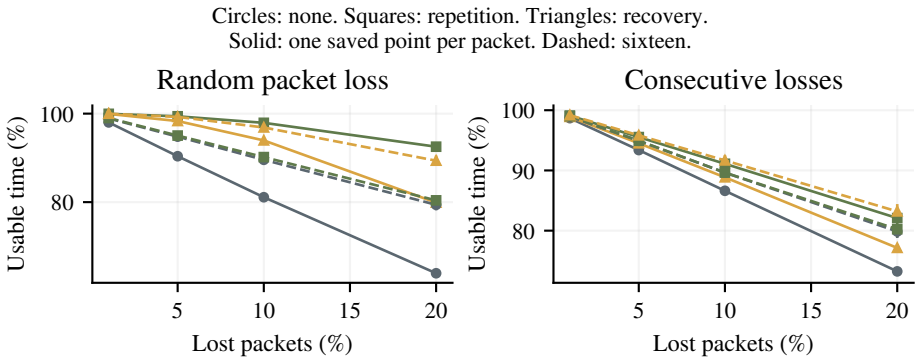


Fig. 5. More recovery information preserves usable time at an added byte cost under random and consecutive packet losses

Table 10. The mathematical guarantees, observed results and remaining questions have different scopes

Evidence	What it supports
Mathematical proofs	Limits on rounding changes, the nonnegative-power condition, clipping area and decisive exact checks.
These experiments	Passing-record counts, file sizes, failure causes, local repairs and small paired differences.
Still to be validated	Sensor accuracy, future-flight reliability, onboard processing time and radio-energy savings.

References

1. Pelkonen, T., Franklin, S., Teller, J., et al.: Gorilla: A Fast, Scalable, In-Memory Time Series Database. Proc. VLDB Endow. 8(12), 1816–1827 (2015).
2. Blalock, D., Madden, S., Guttag, J.: Sprintz: Time Series Compression for the Internet of Things. Proc. ACM Interact. Mob. Wearable Ubiquitous Technol. 2(3), 1–23 (2018).
3. Liakos, P., Papakonstantinopoulou, K., Kotidis, Y.: Chimp: Efficient Lossless Floating Point Compression for Time Series Databases. Proc. VLDB Endow. 15(11), 3058–3070 (2022).
4. Di, S., Cappello, F.: Fast Error-Bounded Lossy HPC Data Compression with SZ. Proc. IEEE IPDPS, 730–739 (2016).
5. Lindstrom, P.: Fixed-Rate Compressed Floating-Point Arrays. IEEE Trans. Vis. Comput. Graph. 20(12), 2674–2683 (2014).
6. Douglas, D.H., Peucker, T.K.: Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or Its Caricature. Cartographica 10(2), 112–122 (1973).
7. Grutzmacher, F., Beichler, B., Hein, A., Kirste, T., Haubelt, C.: Time and Memory Efficient Online Piecewise Linear Approximation of Sensor Signals. Sensors 18(6), 1672 (2018).

8. Fritsch, F.N., Carlson, R.E.: Monotone Piecewise Cubic Interpolation. SIAM J. Numer. Anal. 17(2), 238–246 (1980).
9. Carmona-Poyato, Á., Fernandez-Garcia, N.L., Madrid-Cuevas, F.J., Duran-Rosal, A.M.: A new approach for optimal offline time-series segmentation with error bound guarantee. Pattern Recognition 115, 107917 (2021).
10. Goldberg, D.: What every computer scientist should know about floating-point arithmetic. ACM Comput. Surv. 23(1), 5–48 (1991).
11. IEEE Standard for Floating-Point Arithmetic. IEEE (2019).
12. Garcia-Gascon, C., Bas-Bolufer, J., Castello-Pedrero, P., Garcia-Manrique, J.A.: An open benchmark dataset for machine learning and intelligent trajectory optimization in fixed-wing unmanned aerial systems. Sci. Data 13(1), 364 (2026).
13. Garcia Gascon, C.: Fixed-Wing UAS Telemetry Benchmark (IDF_DS): 240 Flights for ML and Intelligent Trajectory Optimization. Zenodo (2025). https://doi.org/10.5281/zenodo.16992976
14. Rodrigues, T.A., Patrikar, J., Choudhry, A., et al.: In-flight positional and energy use data set of a DJI Matrice 100 quadcopter for small package delivery. Sci. Data 8(1), 155 (2021).
15. Rodrigues, T.A., Patrikar, J., Choudhry, A., et al.: Data Collected with Package Delivery Quadcopter Drone. Carnegie Mellon University (2021). https://doi.org/10.1184/R1/12683453
16. Lee, M., Hu, Y., Zhu, Y., et al.: AMOVFLY: Enabling Advanced UAV Modeling With the Comprehensive Flight Status Dataset. IEEE Trans. Intell. Transp. Syst., 1–16 (2026).
17. Oettershagen, P., Stastny, T., Melzer, A., et al.: AtlantikSolar 81-hour solar-powered flight dataset. ETH Zurich (2015). https://doi.org/10.3929/ethz-c-000787517
18. Elmeleegy, H., Elmagarmid, A.K., Cecchet, E., Aref, W.G., Zwaenepoel, W.: Online Piece-wise Linear Approximation of Numerical Streams with Precision Guarantees. Proc. VLDB Endow. 2(1), 145–156 (2009).
19. Collet, Y., Kucherawy, M.: Zstandard Compression and the application/zstd Media Type. RFC 8878 (2021). https://doi.org/10.17487/RFC8878
20. Efron, B.: Bootstrap Methods: Another Look at the Jackknife. Ann. Stat. 7(1), 1–26 (1979).
21. Holm, S.: A Simple Sequentially Rejective Multiple Test Procedure. Scand. J. Stat. 6(2), 65–70 (1979).
22. Gilbert, E.N.: Capacity of a Burst-Noise Channel. Bell Syst. Tech. J. 39(5), 1253–1265 (1960).
23. Elliott, E.O.: Estimates of Error Rates for Codes on Burst-Noise Channels. Bell Syst. Tech. J. 42(5), 1977–1997 (1963).
24. Liang, X., Zhao, K., Di, S., et al.: SZ3: A Modular Framework for Composing Prediction-Based Error-Bounded Lossy Compressors. IEEE Trans. Big Data 9(2), 485–498 (2023).
25. Kitsios, X., Liakos, P., Papakonstantinopoulou, K., Kotidis, Y.: Sim-Piece: Highly Accurate Piecewise Linear Approximation through Similar Segment Merging. Proc. VLDB Endow. 16(8), 1910–1922 (2023).
26. Farouki, R.T.: The Bernstein polynomial basis: A centennial retrospective. Comput. Aided Geom. Des. 29(6), 379–419 (2012).
27. WebAssembly Community Group: WebAssembly Core Specification, Binary Format, Integer values. https://webassembly.github.io/spec/core/binary/values.html (accessed 2026-11-01).